Socket Programming Practice Assignments
PCCSL507 – Computer Networks Laboratory
Department of Computer Science & Engineering

Submitted by:

Rose Soyuse

S5 CSE A

Roll number: 54

Socket Programming Practice Assignments

Question 1

Hello Client-Server (Beginner) Problem Statement Develop a TCP client-server application where: • The client sends the message ”Hello Server”. • The server receives the message and displays it. • The server replies with ”Hello Client”. • The client displays the received response. Skills Practiced • socket() • bind() • listen() • accept() • connect() • send() • recv()

Answer:

Client code:

client.c > ...

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <winsock2.h>
#include <ws2tcpip.h>

#define PORT 8080

int main()
{
    WSADATA wsa;
    SOCKET sockfd;
    struct sockaddr_in serverAddr;
    char buffer[1024];

    // Initialize Winsock
    if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0)
    {
        printf("WSAStartup Failed\n");
        return 1;
    }

    // Create Socket
    sockfd = socket(AF_INET, SOCK_STREAM, 0);

    if (sockfd == INVALID_SOCKET)
    {
        printf("Socket Creation Failed\n");
        WSACleanup();
```

```
        return 1;
    }

    printf("Socket Created Successfully...\n");

    // Server Address
    serverAddr.sin_family = AF_INET;
    serverAddr.sin_port = htons(PORT);

    inet_pton(AF_INET, "127.0.0.1", &serverAddr.sin_addr);

    // Connect
    if (connect(sockfd,
                (struct sockaddr *)&serverAddr,
                sizeof(serverAddr)) == SOCKET_ERROR)
    {
        printf("Connection Failed\n");
        closesocket(sockfd);
        WSACleanup();
        return 1;
    }

    printf("Connected to Server...\n");

    // Send message
    strcpy(buffer, "Hello Server");
```

```
    strcpy(buffer, "Hello Server");

    send(sockfd, buffer, strlen(buffer) + 1, 0);

    printf("Message Sent : %s\n", buffer);

    // Receive reply
    recv(sockfd, buffer, sizeof(buffer), 0);

    printf("Reply from Server : %s\n", buffer);

    // Close socket
    closesocket(sockfd);

    printf("Connection Closed...\n");

    WSACleanup();

    return 0;
}
```

Server code:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <winsock2.h>

#define PORT 8080

int main()
{
    WSADATA wsa;
    SOCKET serverSocket, clientSocket;
    struct sockaddr_in serverAddr;
    char buffer[1024];

    // Initialize Winsock
    if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0)
    {
        printf("WSAStartup Failed\n");
        return 1;
    }

    // Create Socket
    serverSocket = socket(AF_INET, SOCK_STREAM, 0);

    if (serverSocket == INVALID_SOCKET)
    {
        printf("Socket Creation Failed\n");
        WSACleanup();
        return 1;
    }

    printf("Socket Created Successfully...\n");

    // Server Address
    serverAddr.sin_family = AF_INET;
    serverAddr.sin_addr.s_addr = INADDR_ANY;
    serverAddr.sin_port = htons(PORT);

    // Bind
    if (bind(serverSocket,(struct sockaddr *)&serverAddr,
    sizeof(serverAddr)) == SOCKET_ERROR)
    {
        printf("Bind Failed\n");
        closesocket(serverSocket);
        WSACleanup();
        return 1;
    }

    printf("Bind Successful...\n");

    // Listen
    listen(serverSocket, 5);

    printf("Waiting for Client...\n");

    // Accept
    clientSocket = accept(serverSocket, NULL, NULL);

```

```c
    if (clientSocket == INVALID_SOCKET)
    {
        printf("Accept Failed\n");
        closesocket(serverSocket);
        WSACleanup();
        return 1;
    }
    printf("Client Connected...\n");

    // Receive message
    recv(clientSocket, buffer, sizeof(buffer), 0);

    printf("Client Says : %s\n", buffer);
    // Send reply
    strcpy(buffer, "Hello Client");

    send(clientSocket, buffer, strlen(buffer) + 1, 0);

    printf("Reply Sent...\n");
    // Close sockets
    closesocket(clientSocket);
    closesocket(serverSocket);

    printf("Connection Closed...\n");
    WSACleanup();
return 0;
}
```

Output server

```
C:\Users\Soyuse Peter\OneDrive\Desktop\CN_Lab>gcc server.c -o server -lws2_
32

C:\Users\Soyuse Peter\OneDrive\Desktop\CN_Lab>server
Socket Created Successfully...
Bind Successful...
Waiting for Client...
Client Connected...
Client Says : Hello Server
Reply Sent...
Connection Closed...

C:\Users\Soyuse Peter\OneDrive\Desktop\CN_Lab>
```

Output Client

```
C:\Users\Soyuse Peter\OneDrive\Desktop\CN_Lab>gcc client.c -o client -lws2_
32

C:\Users\Soyuse Peter\OneDrive\Desktop\CN_Lab>client
Socket Created Successfully...
Connected to Server...
Message Sent : Hello Server
Reply from Server : Hello Client
Connection Closed...

C:\Users\Soyuse Peter\OneDrive\Desktop\CN_Lab>
```

Question 2

Arithmetic Server Problem Statement Develop a TCP client-server application where: • The client inputs two integers. • The client sends both numbers to the server. • The server calculates:– Addition – Subtraction – Multiplication – Division • The server sends all results back. • The client displays the received results. Skills Practiced • Integer Communication • Arithmetic Operations • TCP Socket Programming

Answer:

Server code

server.c > main()

```c
#include <stdio.h>
#include <stdlib.h>
#include <winsock2.h>

#define PORT 8080

int main()
{
    WSADATA wsa;
    SOCKET serverSocket, clientSocket;
    struct sockaddr_in serverAddr;

    int num1, num2;
    int addition, subtraction, multiplication;
    float division;

    // Initialize Winsock
    if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0)
    {
        printf("WSAStartup Failed\n");
        return 1;
    }

    // Create Socket
    serverSocket = socket(AF_INET, SOCK_STREAM, 0);

    if (serverSocket == INVALID_SOCKET)
    {
        printf("Socket Creation Failed\n");
```

server.c > main()

```c
    if (serverSocket == INVALID_SOCKET)
    {
        printf("Socket Creation Failed\n");
        WSACleanup();
        return 1;
    }

    printf("Socket Created Successfully...\n");

    // Server Address
    serverAddr.sin_family = AF_INET;
    serverAddr.sin_addr.s_addr = INADDR_ANY;
    serverAddr.sin_port = htons(PORT);

    // Bind
    if (bind(serverSocket,
             (struct sockaddr *)&serverAddr,
             sizeof(serverAddr)) == SOCKET_ERROR)
    {
        printf("Bind Failed\n");
        closesocket(serverSocket);
        WSACleanup();
        return 1;
    }

    printf("Bind Successful...\n");

    // Listen
    listen(serverSocket, 5);
```

```c
    listen(serverSocket, 5);

    printf("Waiting for Client...\n");

    // Accept
    clientSocket = accept(serverSocket, NULL, NULL);

    if (clientSocket == INVALID_SOCKET)
    {
        printf("Accept Failed\n");
        closesocket(serverSocket);
        WSACleanup();
        return 1;
    }

    printf("Client Connected...\n");

    // Receive numbers
    recv(clientSocket, (char *)&num1, sizeof(num1), 0);
    recv(clientSocket, (char *)&num2, sizeof(num2), 0);

    printf("Received Numbers : %d and %d\n", num1, num2);

    // Perform calculations
    addition = num1 + num2;
    subtraction = num1 - num2;
    multiplication = num1 * num2;
```

```
82
83      if (num2 != 0)
84          division = (float)num1 / num2;
85      else
86          division = 0;
87
88      // Send results
89      send(clientSocket, (char *)&addition, sizeof(addition), 0);
90      send(clientSocket, (char *)&subtraction, sizeof(subtraction), 0);
91      send(clientSocket, (char *)&multiplication, sizeof(multiplication), 0
92      send(clientSocket, (char *)&division, sizeof(division), 0);
93
94      printf("Results Sent Successfully...\n");
95
96      // Close
97      closesocket(clientSocket);
98      closesocket(serverSocket);
99
100     printf("Connection Closed...\n");
101
102     WSACleanup();
103
104     return 0;
105 }
```

Client code:

client.c > main()

```
1   #include <stdio.h>
2   #include <stdlib.h>
3   #include <winsock2.h>
4
5   #define PORT 8080
6
7   int main()
8   {
9       WSADATA wsa;
10      SOCKET sockfd;
11      struct sockaddr_in serverAddr;
12
13      int num1, num2;
14      int addition, subtraction, multiplication;
15      float division;
16
17      // Initialize Winsock
18      if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0)
19      {
20          printf("WSAStartup Failed\n");
21          return 1;
22      }
23
24      // Create Socket
25      sockfd = socket(AF_INET, SOCK_STREAM, 0);
26
27      if (sockfd == INVALID_SOCKET)
28      {
29          printf("Socket Creation Failed\n");
```

```
29          printf("Socket Creation Failed\n");
30          WSACleanup();
31          return 1;
32      }
33
34      printf("Socket Created Successfully...\n");
35
36
37
38      serverAddr.sin_port = htons(PORT);
39      serverAddr.sin_addr.s_addr = inet_addr("127.0.0.1");
40
41      // Connect
42      if (connect(sockfd,
43                  (struct sockaddr *)&serverAddr,
44                  sizeof(serverAddr)) == SOCKET_ERROR)
45      {
46          printf("Connection Failed\n");
47          closesocket(sockfd);
48          WSACleanup();
49          return 1;
50      }
51
52      printf("Connected to Server...\n");
53
54      // Input numbers
55      printf("Enter first integer : ");
56      scanf("%d", &num1);
57
```

```
58      printf("Enter second integer : ");
59      scanf("%d", &num2);
60
61      // Send numbers
62      send(sockfd, (char *)&num1, sizeof(num1), 0);
63      send(sockfd, (char *)&num2, sizeof(num2), 0);
64
65      printf("Numbers Sent Successfully...\n");
66
67      // Receive results
68      recv(sockfd, (char *)&addition, sizeof(addition), 0);
69      recv(sockfd, (char *)&subtraction, sizeof(subtraction), 0);
70      recv(sockfd, (char *)&multiplication, sizeof(multiplication), 0);
71      recv(sockfd, (char *)&division, sizeof(division), 0);
72
73      // Display results
74      printf("\nResults from Server\n");
75      printf("Addition       : %d\n", addition);
76      printf("Subtraction    : %d\n", subtraction);
77      printf("Multiplication : %d\n", multiplication);
78      printf("Division       : %.2f\n", division);
79
80      // Close
81      closesocket(sockfd);
82      printf("\nConnection Closed...\n");
83      WSACleanup();
84      return 0;
85  }
```

Server output

```
Socket Created Successfully...
Bind Successful...
Waiting for Client...
Client Connected...
Received Numbers : 20 and 5
Results Sent Successfully...
Connection Closed...

C:\Users\Soyuse Peter\OneDrive\Desktop\Q2_Arithmetic>
```

Client output

```
Socket Created Successfully...
Connected to Server...
Enter first integer : 20
Enter second integer : 5
Numbers Sent Successfully...

Results from Server
Addition       : 25
Subtraction    : 15
Multiplication : 100
Division       : 4.00

Connection Closed...
```

Question 3

String Processing Server Problem Statement Develop a TCP client-server application where: • The client inputs a string. • The server performs the following operations: – Find the string length – Convert the string to uppercase – Reverse the string • The server sends the processed results. • The client displays all outputs. Skills Practiced • Character Arrays • String Manipulation • TCP Communication

Answer:

Server code

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <winsock2.h>

#define PORT 8080

int main()
{
    WSADATA wsa;
    SOCKET serverSocket, clientSocket;
    struct sockaddr_in serverAddr;

    char buffer[1024];
    char uppercase[1024];
    char reversed[1024];
    int length;

    if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0)
    {
        printf("WSAStartup Failed\n");
        return 1;
    }

    serverSocket = socket(AF_INET, SOCK_STREAM, 0);

    if (serverSocket == INVALID_SOCKET)
    {
        printf("Socket Creation Failed\n");
        WSACleanup();
        return 1;
    }

    printf("Socket Created Successfully...\n");

    serverAddr.sin_family = AF_INET;
    serverAddr.sin_addr.s_addr = INADDR_ANY;
    serverAddr.sin_port = htons(PORT);

    if (bind(serverSocket,
             (struct sockaddr *)&serverAddr,
             sizeof(serverAddr)) == SOCKET_ERROR)
    {
        printf("Bind Failed\n");
        closesocket(serverSocket);
        WSACleanup();
        return 1;
    }

    printf("Bind Successful...\n");

    listen(serverSocket, 5);

    printf("Waiting for Client...\n");

    clientSocket = accept(serverSocket, NULL, NULL);
```

```
    if (clientSocket == INVALID_SOCKET)
    {
        printf("Accept Failed\n");
        closesocket(serverSocket);
        WSACleanup();
        return 1;
    }

    printf("Client Connected...\n");

    recv(clientSocket, buffer, sizeof(buffer), 0);

    printf("Received String : %s\n", buffer);

    // Find length
    length = strlen(buffer);

    // Convert to uppercase
    strcpy(uppercase, buffer);

    for (int i = 0; uppercase[i] != '\0'; i++)
    {
        uppercase[i] = toupper(uppercase[i]);
    }

    // Reverse string
    for (int i = 0; i < length; i++)
    {
        reversed[i] = buffer[length - 1 - i];
    }

    reversed[length] = '\0';

    // Send results
    send(clientSocket, (char *)&length, sizeof(length), 0);
    send(clientSocket, uppercase, strlen(uppercase) + 1, 0);
    send(clientSocket, reversed, strlen(reversed) + 1, 0);

    printf("Processed Results Sent Successfully...\n");

    closesocket(clientSocket);
    closesocket(serverSocket);

    printf("Connection Closed...\n");

    WSACleanup();

    return 0;
}
```

Client code:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <winsock2.h>

#define PORT 8080

int main()
{
    WSADATA wsa;
    SOCKET sockfd;
    struct sockaddr_in serverAddr;

    char buffer[1024];
    int length;

    if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0)
    {
        printf("WSAStartup Failed\n");
        return 1;
    }

    sockfd = socket(AF_INET, SOCK_STREAM, 0);

    if (sockfd == INVALID_SOCKET)
    {
        printf("Socket Creation Failed\n");
        WSACleanup();
        return 1;
    }

    printf("Socket Created Successfully...\n");

    serverAddr.sin_family = AF_INET;
    serverAddr.sin_port = htons(PORT);
    serverAddr.sin_addr.s_addr = inet_addr("127.0.0.1");

    if (connect(sockfd,
                (struct sockaddr *)&serverAddr,
                sizeof(serverAddr)) == SOCKET_ERROR)
    {
        printf("Connection Failed\n");
        closesocket(sockfd);
        WSACleanup();
        return 1;
    }

    printf("Connected to Server...\n");

    printf("Enter a string : ");
    fgets(buffer, sizeof(buffer), stdin);

    // Remove newline
    buffer[strcspn(buffer, "\n")] = '\0';

    send(sockfd, buffer, strlen(buffer) + 1, 0);

    printf("String Sent Successfully...\n");
```

```c
    // Receive results
    recv(sockfd, (char *)&length, sizeof(length), 0);

    recv(sockfd, buffer, sizeof(buffer), 0);

    char uppercase[1024];
    strcpy(uppercase, buffer);

    recv(sockfd, buffer, sizeof(buffer), 0);

    char reversed[1024];
    strcpy(reversed, buffer);

    printf("\nResults from Server\n");
    printf("String Length : %d\n", length);
    printf("Uppercase     : %s\n", uppercase);
    printf("Reversed      : %s\n", reversed);

    closesocket(sockfd);

    printf("\nConnection Closed...\n");

    WSACleanup();

    return 0;
}
```

Server output:

```
C:\Users\Soyuse Peter\OneDrive\Desktop\Q2_Arithmetic>gcc server.c -o server
 -lws2_32

C:\Users\Soyuse Peter\OneDrive\Desktop\Q2_Arithmetic>server
Socket Created Successfully...
Bind Successful...
Waiting for Client...
Client Connected...
Received String : Hello World
Processed Results Sent Successfully...
Connection Closed...
```

Client output:

```
C:\Users\Soyuse Peter\OneDrive\Desktop\Q2_Arithmetic>client
Socket Created Successfully...
Connected to Server...
Enter a string : Hello World
String Sent Successfully...

Results from Server
String Length : 11
Uppercase     : HELLO WORLD
Reversed      : dlroW olleH

Connection Closed...
```

Question 4

Array Processing Server Problem Statement Develop a TCP client-server application where: • The client inputs N integers. • The array is sent to the server. • The server computes: – Sum – Average – Maximum Element – Minimum Element • The server sends the results. • The client displays them. Skills Practiced • Arrays • Loops • TCP Socket Programming

Answer:

Server code:

server.c > main()

```c
#include <stdio.h>
#include <stdlib.h>
#include <winsock2.h>

#define PORT 8080
#define MAX 100

int main()
{
    WSADATA wsa;
    SOCKET serverSocket, clientSocket;
    struct sockaddr_in serverAddr;

    int n;
    int arr[MAX];
    int sum = 0;
    int maximum, minimum;
    float average;

    // Initialize Winsock
    if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0)
    {
        printf("WSAStartup Failed\n");
        return 1;
    }

    // Create Socket
    serverSocket = socket(AF_INET, SOCK_STREAM, 0);

```

```c
    if (serverSocket == INVALID_SOCKET)
    {
        printf("Socket Creation Failed\n");
        WSACleanup();
        return 1;
    }

    printf("Socket Created Successfully...\n");

    // Server Address
    serverAddr.sin_family = AF_INET;
    serverAddr.sin_addr.s_addr = INADDR_ANY;
    serverAddr.sin_port = htons(PORT);

    // Bind
    if (bind(serverSocket,
             (struct sockaddr *)&serverAddr,
             sizeof(serverAddr)) == SOCKET_ERROR)
    {
        printf("Bind Failed\n");
        closesocket(serverSocket);
        WSACleanup();
        return 1;
    }

    printf("Bind Successful...\n");

    // Listen
    listen(serverSocket, 5);
```

```c
    printf("Waiting for Client...\n");

    // Accept
    clientSocket = accept(serverSocket, NULL, NULL);

    if (clientSocket == INVALID_SOCKET)
    {
        printf("Accept Failed\n");
        closesocket(serverSocket);
        WSACleanup();
        return 1;
    }

    printf("Client Connected...\n");

    // Receive N
    recv(clientSocket, (char *)&n, sizeof(n), 0);

    // Receive array
    recv(clientSocket, (char *)arr, sizeof(int) * n, 0);

    printf("Received Array : ");

    for (int i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }

    printf("\n");
```

```
    // Calculate sum
    for (int i = 0; i < n; i++)
    {
        sum += arr[i];
    }

    // Find maximum and minimum
    maximum = arr[0];
    minimum = arr[0];

    for (int i = 1; i < n; i++)
    {
        if (arr[i] > maximum)
            maximum = arr[i];

        if (arr[i] < minimum)
            minimum = arr[i];
    }

    average = (float)sum / n;

    // Send results
    send(clientSocket, (char *)&sum, sizeof(sum), 0);
    send(clientSocket, (char *)&average, sizeof(average), 0);
    send(clientSocket, (char *)&maximum, sizeof(maximum), 0);
    send(clientSocket, (char *)&minimum, sizeof(minimum), 0);

    printf("Results Sent Successfully...\n");
```

```
    // Close
    closesocket(clientSocket);
    closesocket(serverSocket);

    printf("Connection Closed...\n");

    WSACleanup();

    return 0;
}
```

Server output:

```
Socket Created Successfully...
Bind Successful...
Waiting for Client...
Client Connected...
Received Array : 5 10 20 30 40 50
Results Sent Successfully...
Connection Closed...
```

Client output:

```
Socket Created Successfully...
Connected to Server...
Enter number of elements : 6
Enter 6 integers:
5
10
20
30
40
50
Array Sent Successfully...

Results from Server
Sum     : 155
```

```
Results from Server
Sum     : 155
Average : 25.83
Maximum : 50
Minimum : 5

Connection Closed...
```

Question 5 – Matrix Analyzer Problem Statement Develop a TCP client-server application where: • The client inputs a square matrix of order N. • The matrix is sent to the server. • The server determines: – Matrix Type – Sum of Matrix Elements – Trace of Matrix • The server sends the results. • The client displays the received information. Matrix Types • Diagonal Matrix • Upper Triangular Matrix • Lower Triangular Matrix • General Matrix Skills Practiced • Two-Dimensional Arrays • Matrix Operations • TCP Socket Programming

Answer:

Server code:

server.c > main()

```c
#include <stdio.h>
#include <stdlib.h>
#include <winsock2.h>

#define PORT 8080
#define MAX 100

int main()
{
    WSADATA wsa;
    SOCKET serverSocket, clientSocket;
    struct sockaddr_in serverAddr;

    int n;
    int matrix[MAX][MAX];

    int sum = 0;
    int trace = 0;

    int diagonal = 1;
    int upper = 1;
    int lower = 1;

    char result[100];

    // Initialize Winsock
    if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0)
    {
        printf("WSAStartup Failed\n");
        return 1;
    }

    // Create Socket
    serverSocket = socket(AF_INET, SOCK_STREAM, 0);

    if (serverSocket == INVALID_SOCKET)
    {
        printf("Socket Creation Failed\n");
        WSACleanup();
        return 1;
    }

    printf("Socket Created Successfully...\n");

    // Server Address
    serverAddr.sin_family = AF_INET;
    serverAddr.sin_addr.s_addr = INADDR_ANY;
    serverAddr.sin_port = htons(PORT);

    // Bind
    if (bind(serverSocket,
             (struct sockaddr *)&serverAddr,
             sizeof(serverAddr)) == SOCKET_ERROR)
    {
        printf("Bind Failed\n");
        closesocket(serverSocket);
        WSACleanup();
        return 1;
```

```
    }

    printf("Bind Successful...\n");

    // Listen
    listen(serverSocket, 5);

    printf("Waiting for Client...\n");

    // Accept
    clientSocket = accept(serverSocket, NULL, NULL);

    if (clientSocket == INVALID_SOCKET)
    {
        printf("Accept Failed\n");
        closesocket(serverSocket);
        WSACleanup();
        return 1;
    }

    printf("Client Connected...\n");

    // Receive matrix order
    recv(clientSocket, (char *)&n, sizeof(n), 0);

    // Receive matrix
    recv(clientSocket,
         (char *)matrix,
         sizeof(int) * n * n,
         0);

    printf("Received Matrix:\n");

    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }

    // Analyze matrix
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            sum += matrix[i][j];

            if (i == j)
                trace += matrix[i][j];

            // For diagonal matrix
            if (i != j && matrix[i][j] != 0)
                diagonal = 0;

            // For upper triangular matrix
            if (i > j && matrix[i][j] != 0)
```

```
            if (i > j && matrix[i][j] != 0)
                upper = 0;

            // For lower triangular matrix
            if (i < j && matrix[i][j] != 0)
                lower = 0;
        }
    }

    // Determine matrix type
    if (diagonal)
        strcpy(result, "Diagonal Matrix");
    else if (upper)
        strcpy(result, "Upper Triangular Matrix");
    else if (lower)
        strcpy(result, "Lower Triangular Matrix");
    else
        strcpy(result, "General Matrix");

    // Send results
    send(clientSocket,
         result,
         strlen(result) + 1,
         0);

    send(clientSocket,
         (char *)&sum,
         sizeof(sum),
         0);
```

```

    send(clientSocket,
         (char *)&trace,
         sizeof(trace),
         0);

    printf("Matrix Type : %s\n", result);
    printf("Sum : %d\n", sum);
    printf("Trace : %d\n", trace);

    printf("Results Sent Successfully...\n");

    // Close
    closesocket(clientSocket);
    closesocket(serverSocket);

    printf("Connection Closed...\n");

    WSACleanup();

    return 0;
}
```

Client code:

```c
#include <stdio.h>
#include <stdlib.h>
#include <winsock2.h>

#define PORT 8080
#define MAX 100

int main()
{
    WSADATA wsa;
    SOCKET sockfd;
    struct sockaddr_in serverAddr;

    int n;
    int matrix[MAX][MAX];

    char result[100];

    int sum;
    int trace;

    // Initialize Winsock
    if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0)
    {
        printf("WSAStartup Failed\n");
        return 1;
    }

    // Create Socket
    sockfd = socket(AF_INET, SOCK_STREAM, 0);

    if (sockfd == INVALID_SOCKET)
    {
        printf("Socket Creation Failed\n");
        WSACleanup();
        return 1;
    }

    printf("Socket Created Successfully...\n");

    // Server Address
    serverAddr.sin_family = AF_INET;
    serverAddr.sin_port = htons(PORT);
    serverAddr.sin_addr.s_addr = inet_addr("127.0.0.1");

    // Connect
    if (connect(sockfd,
                (struct sockaddr *)&serverAddr,
                sizeof(serverAddr)) == SOCKET_ERROR)
    {
        printf("Connection Failed\n");
        closesocket(sockfd);
        WSACleanup();
        return 1;
    }

    printf("Connected to Server...\n");

```

```
    // Input matrix order
    printf("Enter Matrix Order : ");
    scanf("%d", &n);

    // Input matrix
    printf("Enter Matrix Elements:\n");

    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            scanf("%d", &matrix[i][j]);
        }
    }

    // Display matrix
    printf("\nMatrix:\n");

    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }

    // Send matrix order
    send(sockfd,
```

```
         (char *)&n,
         sizeof(n),
         0);

    // Send matrix
    send(sockfd,
         (char *)matrix,
         sizeof(int) * n * n,
         0);

    printf("\nMatrix Sent Successfully...\n");

    // Receive results
    recv(sockfd,
         result,
         sizeof(result),
         0);

    recv(sockfd,
         (char *)&sum,
         sizeof(sum),
         0);

    recv(sockfd,
         (char *)&trace,
         sizeof(trace),
         0);

    // Display results
```

```c
    // Display results
    printf("\nResults from Server\n");
    printf("Matrix Type : %s\n", result);
    printf("Sum         : %d\n", sum);
    printf("Trace       : %d\n", trace);

    // Close
    closesocket(sockfd);

    printf("\nConnection Closed...\n");

    WSACleanup();

    return 0;
}
```

Server output:

```
Socket Created Successfully...
Bind Successful...
Waiting for Client...
Client Connected...
Received Matrix:
1 0 0
0 0 0
0 0 0
Matrix Type : Diagonal Matrix
Sum : 1
Trace : 1
Results Sent Successfully...
Connection Closed...
```

Client output:

```
Socket Created Successfully...
Connected to Server...
Enter Matrix Order : 3
Enter Matrix Elements:
1 0 0 0 2 0 0 0 3

Matrix:
1 0 0
0 2 0
0 0 3

Matrix Sent Successfully...

Results from Server
Matrix Type : Diagonal Matrix
Sum         : 1
Trace       : 1

Connection Closed...
```